

Trabalho 2 de MC458 - Pizzada do IC

Antônio Queiroz Manetta - RA:231565

25 de Junho de 2023

1 Introdução

O seguinte projeto tem por objetivo fornecer a solução ótima para o problema da pizzada do IC. O problema consiste no seguinte: Temos que assar N pizzas em um tempo T , sendo que cada pizza tem associado a ela um sabor inicial, s_i , um tempo para assar, t_i e um coeficiente de perda de sabor por tempo, r_i . Não necessariamente vamos conseguir assar todas as pizzas, e queremos maximizar o sabor total, somando os valores do sabor de cada pizza no momento em que terminam de ser assadas. Devemos por último, retornar o valor dessa soma final de sabores que obteremos com o algoritmo.

2 A corretude do algoritmo

Primeiramente, observamos que para resolver o problema, podemos utilizar um algoritmo de programação dinâmica, mais especificamente, este é um problema da mochila binária, onde escolhemos que itens levar com base no valor associado a eles, mas com algumas diferenças a serem destacadas: neste caso, a ordem dos itens importa, pois o valor de cada item é variável conforme o tempo.

Tendo isso em vista, a primeira parte do algoritmo trata de ordenar o vetor de pizzas de forma a facilitar o uso da mochila. Para isso, precisamos analisar primeiro quais itens devemos dar preferência na fila, e quais itens devemos deixar para depois.

Podemos enxergar facilmente que, para se obter um valor do sabor final máximo, as pizzas que perdem sabor mais rapidamente devem ser assadas primeiro, e as que perdem mais devagar podem ser assadas posteriormente, pois o tempo passado até terminar de assá-la interfere menos no resultado final. Assim, o parâmetro que vai balizar a ordenação será a razão r_i/t_i , sendo que essa razão será decrescente. Dessa forma, garantimos que a pizza que perde sabor mais rápido, demora menos tempo para ser feita e faz parte da solução ótima do problema seja preparada primeiro, e assim por diante.

Tendo isso em mente, utiliza-se o algoritmo mergesort de ordenação de vetores utilizando como parâmetro a razão r_i/t_i , e assim podemos seguir para o problema da mochila.

Em sequência, utilizamos a tabela da mochila binária para achar a solução ótima. Isto é, se existe uma combinação de pizzas que traz o valor ótimo para o sabor, essa combinação será encontrada na sequência em que foi ordenado o vetor de acordo com o algoritmo anterior.

O algoritmo cria uma tabela de Z de $N+1$ por $T+1$, e cada um dos elementos $Z(i,j)$ dessa matriz será a solução ótima para a combinação das i primeiras pizzas com j de tempo disponível. Assim, quando por recorrência, utilizamos um valor da tabela anterior, estamos buscando um valor ótimo para aquele subproblema, ou melhor, para aquela subestrutura. A sequência do algoritmo vai inicializar a primeira linha e a primeira coluna como 0, pois exprimem o resultado ótimo para nenhuma pizza e o resultado ótimo para tempo nulo, respectivamente. Em sequência, verificamos se a pizza i pode ser feita no tempo j , e compararmos o valor da pizza i no tempo j com o valor da solução ótima para as $i-1$ pizzas, ou seja, a solução ótima sem contar com a pizza i , e escolher o maior. Depois, fazer uma comparação entre esse valor e o valor da solução para a linha i com tempo $j-1$, assim cobrimos a possibilidade de a pizza já estar com valor ótimo em um tempo anterior, e depois consultar se esse valor ótimo é melhor do que a combinação da pizza i com o resto, na ordem em que foram colocadas na tabela. Para isso, comparamos a solução ótima encontrada até o momento e o valor da pizza i no tempo j mais a solução ótima das $i-1$ pizzas no tempo $j - t_i$. Isso exprime a possibilidade de a solução ser melhor colocando a pizza i depois

de uma certa combinação das $i-1$ pizzas. Caso a pizza i não possa ser feita em j de tempo, o algoritmo considera a solução ótima das i primeiras pizzas com j de tempo como a solução das $i-1$ pizzas em tempo j . Dessa forma, garantimos que a solução ótima do problema seja encontrada até no máximo a última iteração do problema, sem ter que calcular algum processo mais de uma vez.

3 A Implementação do Código

Para implementarmos o código, criamos uma função mergesort, que ordena as pizzas. Depois, chamamos a função solve(pizzas, N, T), que resolverá o problema retornando o maior valor possível para o sabor final. A função solve primeiro cria matriz de valores ótimos Z de $N+1$ por $T+1$, alocada dinamicamente, e inicializa a primeira linha e a primeira coluna como zeros. Em seguida, criamos dois laços aninhados e percorremos a matriz em N e em T achando os valores ótimos das combinações possíveis, assim como no problema da mochila. Para cada novo valor na matriz Z , devemos comparar com a linha anterior, com a coluna anterior, e com uma possível combinação entre a solução ótima da linha anterior para um tempo $j - t_i$ e o valor da pizza i no tempo j , sempre selecionando o maior valor possível.

O código após a inicialização da matriz Z fica da seguinte forma:

```
for (int i=1; i< N+1; i++){ # varrer todas as pizzas
    for (int j=1; j< T+1; j++){ # varrer todos os tempos
        if (j>=pizzas[i-1].t){ # se for possível fazer a pizza i:
            Z[i][j]=pizzas[i-1].s-pizzas[i-1].r*j; #valor do sabor da pizza
                i no instante j
            if (Z[i][j]+Z[i-1][j-pizzas[i-1].t]>Z[i][pizzas[i-1].t]){ # se
                a combinação for melhor:
                    Z[i][j]=Z[i][j]+Z[i-1][j-pizzas[i-1].t]; # valor ótimo é a
                        combinação
                }
            else{
                Z[i][j]=Z[i][pizzas[i-1].t]; #caso não seja melhor, o valor
                    ótimo é o valor da pizza i no mpinimo de tempo para
                        ficar pronta
                }
            if (Z[i][j]<Z[i][j-1]){ # se a solução ótima foi pior do que a
                solução com menos tempo:
                    Z[i][j]=Z[i][j-1]; # solução será a maior possível
                }
            if (Z[i][j]<Z[i-1][j]){ # se a solução para a combinação das i
                -1 pizzas é melhor do que a solução encontrada com a pizza i
                    :
                    Z[i][j]=Z[i-1][j]; # solução será a melhor possível.
                }
            }
        }
    }
    else{ # caso não de para fazer a pizza i:
        Z[i][j]=Z[i-1][j]; # solução ótima é a solução ótima para as i
            -1 pizzas.
        }
    }
}
```

Dessa forma, cobrimos todas as possibilidades de soluções ótimas para cada caso, e após terminar a iteração o valor ótimo será o último elemento da matriz, que retornaremos na função solve.

4 Complexidade Temporal do Código

Para analisar a complexidade temporal do algoritmo, precisamos considerar as duas partes que compõem o algoritmo, a ordenação e a função solve. Para a ordenação, foi utilizado um algoritmo de mergesort, que ordena um vetor de tamanho N em $O(N \log(N))$. Em seguida, a função solve criará primeira-

mente dois laços não aninhados, que inicializam as condições de contorno, e depois dois laços aninhados que para cada caso, realizam no máximo um número constante de operações. Portanto, no pior caso, a complexidade de tempo do algoritmo será o tempo para percorrer os laços, que tem $O(N \cdot T)$ operações. Assim, a complexidade temporal da função solve será $O(N \cdot T)$.

Visto que $1 \leq N, T \leq 1000$, podemos supor que no caso mais complicado, ambos N e T valem 1000. Assim, a complexidade temporal do algoritmo ficará $O(N \log(N)) + O(N \cdot T)$, em que a ordem de crescimento é tomado pelo segundo elemento da soma, já que $T = N \Rightarrow T > \log(N)$, assim a complexidade temporal do algoritmo será $O(N \cdot T)$ no caso mais complicado. Caso T seja menor que $\log(N)$ para algum caso, a função mergesort passa a ter uma complexidade de tempo pior do que da função solve, e assim a complexidade de tempo neste caso será $O(N \log(N))$.

Considerando o pior caso, em que $N, T = 1000$, a complexidade de tempo será de $c \cdot 10^6$, sendo c uma constante. Para garantir que não realizemos mais que 10^7 operações, só precisamos garantir que essa constante c seja menor que 10, o que pelo algoritmo é cumprido, dado o número de comparações e de atribuições.

5 Conclusão

Visto que a complexidade de tempo para esse algoritmo é esperado ser $O(N \cdot T)$, submetemos o código e obtemos resultados corretos e dentro do tempo esperado para todos os 20 testes. Isso indica que realmente o algoritmo está correto e resolve o problema de otimização em tempo esperado, e assim justifica a escolha do algoritmo.